DSA0602 DATA HANDLING AND VISUALIZATION

UNIT 4 ACTIVITY 4

NAME:ARCHANA J

REG NUMBER:192324294

DATE:12.08.2026

QUESTIONS:

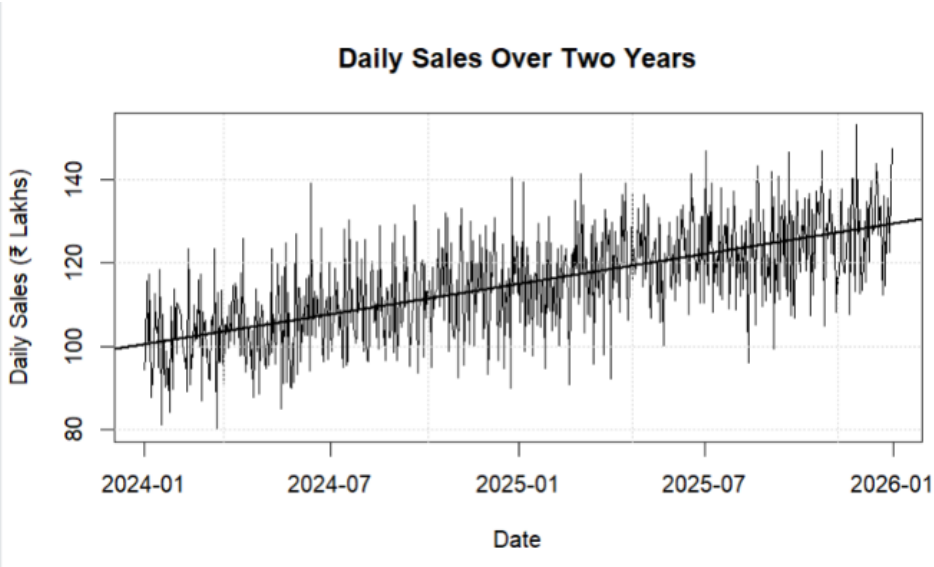
1. A retail chain's finance team wants to understand how daily sales have moved over the past two years before setting next year's budget. Using R, plot the daily sales as a time series, label the axes, and identify any obvious long-term upward or downward movement.

CODE:

```
# Create dates for two years
dates <- seq(
  as.Date("2024-01-01"),
  as.Date("2025-12-31"),
  by = "day"
)

# Generate sample daily sales data
set.seed(123)

sales <- 100 +
  seq(0, 30, length.out = length(dates)) +
  rnorm(length(dates), mean = 0, sd = 10)

# Plot the time series
plot(
```

```
  dates,
  sales,
  type = "l",
  main = "Daily Sales Over Two Years",
  xlab = "Date",
  ylab = "Daily Sales (₹ Lakhs)"
)

# Add grid
grid()

# Add trend line
abline(
  lm(sales ~ as.numeric(dates)),
  lwd = 2
)
```

OUTPUT:



2. An IoT company monitors hourly server temperature at a data center to catch overheating risks early. Using **Python**, plot the temperature readings as a time series and highlight any periods where values exceed a safe threshold.

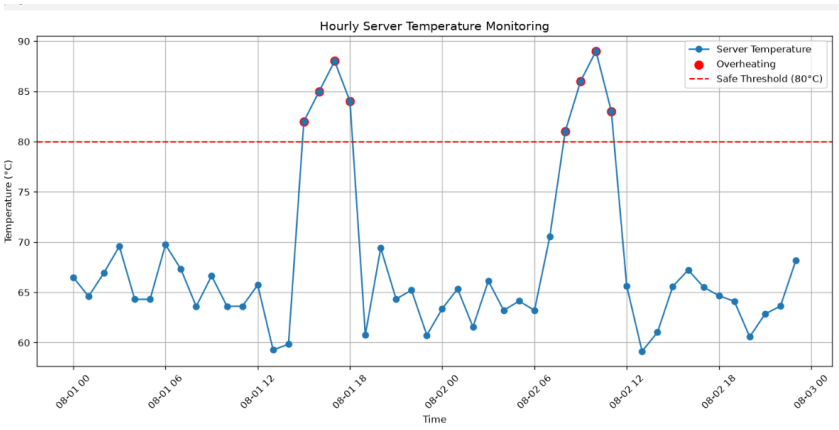
CODE:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt


# Create 48 hours of time data

time = pd.date_range(

   start="2026-08-01 00:00",

   periods=48,

   freq="h"

)


# Simulated server temperature readings

np.random.seed(42)


temperature = 65 + np.random.normal(0, 3, 48)


# Create some overheating periods

temperature[15:19] = [82, 85, 88, 84]

temperature[32:36] = [81, 86, 89, 83]


# Safe temperature threshold

threshold = 80


# Create DataFrame

df = pd.DataFrame({
```

```
    "Time": time,
    "Temperature": temperature
})

# Identify temperatures above the threshold
overheating = df["Temperature"] > threshold

# Plot temperature
plt.figure(figsize=(12, 6))

plt.plot(
    df["Time"],
    df["Temperature"],
    marker="o",
    label="Server Temperature"
)

# Highlight overheating points
plt.scatter(
    df.loc[overheating, "Time"],
    df.loc[overheating, "Temperature"],
    color="red",
    s=70,
    label="Overheating"
)

# Safe threshold line
plt.axhline(
```

```
    y=threshold,
    color="red",
    linestyle="--",
    label="Safe Threshold (80°C)"
)

# Labels and title
plt.title("Hourly Server Temperature Monitoring")
plt.xlabel("Time")
plt.ylabel("Temperature (°C)")

# Grid and legend
plt.grid(True)
plt.legend()

# Rotate date labels
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()

# Display overheating periods
print("Overheating readings:")
print(df[overheating])
```

OUTPUT:



3. A hedge fund analyst is comparing the daily closing prices of three competing tech stocks to decide which one shows steadier growth. Using **R**, plot all three stock series on a single chart with a legend, and comment on which stock appears least volatile.

CODE:

```
# Create dates for 30 trading days
dates <- seq(as.Date("2025-01-01"), by = "day", length.out = 30)


# Create sample stock prices
set.seed(123)


stock_A <- 100 + cumsum(rnorm(30, mean = 0.5, sd = 2))
stock_B <- 100 + cumsum(rnorm(30, mean = 0.5, sd = 1))
stock_C <- 100 + cumsum(rnorm(30, mean = 0.5, sd = 3))


# Plot the three stock series
plot(
  dates,
  stock_A,
  type = "l",
  lwd = 2,
  col = "blue",
  ylim = range(c(stock_A, stock_B, stock_C)),
```

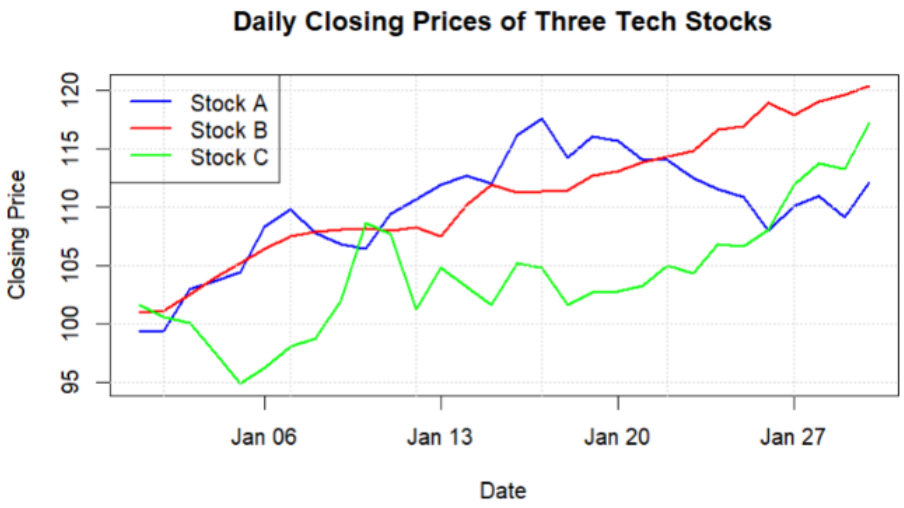
```
  main = "Daily Closing Prices of Three Tech Stocks",
  xlab = "Date",
  ylab = "Closing Price"
)

# Add Stock B
lines(
  dates,
  stock_B,
  col = "red",
  lwd = 2
)

# Add Stock C
lines(
  dates,
  stock_C,
  col = "green",
  lwd = 2
)

# Add legend
legend(
  "topleft",
  legend = c("Stock A", "Stock B", "Stock C"),
  col = c("blue", "red", "green"),
  lty = 1,
  lwd = 2
```

)

Add grid

grid()

OUTPUT:



4. A city's energy board wants to compare monthly electricity consumption across four different neighborhoods to plan targeted conservation campaigns. Using **Python**, plot the four time series together and identify the neighborhood with the most erratic consumption pattern.

CODE:

import pandas as pd

import matplotlib.pyplot as plt

import numpy as np

Create 12 months

months = [

"Jan", "Feb", "Mar", "Apr", "May", "Jun",

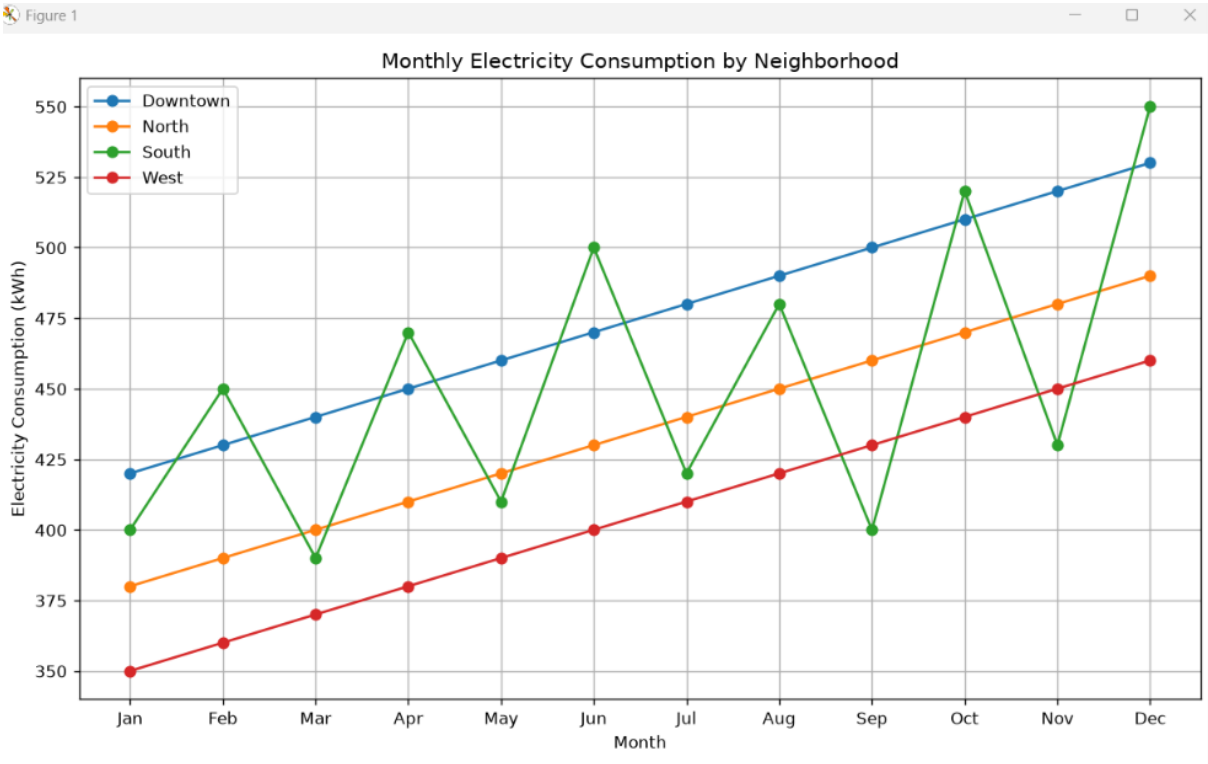
"Jul", "Aug", "Sep", "Oct", "Nov", "Dec"

]

```
# Monthly electricity consumption (kWh)
downtown = [420, 430, 440, 450, 460, 470, 480, 490, 500, 510, 520, 530]

north = [380, 390, 400, 410, 420, 430, 440, 450, 460, 470, 480, 490]

south = [400, 450, 390, 470, 410, 500, 420, 480, 400, 520, 430, 550]

west = [350, 360, 370, 380, 390, 400, 410, 420, 430, 440, 450, 460]

# Create the plot
plt.figure(figsize=(10, 6))

plt.plot(months, downtown, marker="o", label="Downtown")
plt.plot(months, north, marker="o", label="North")
plt.plot(months, south, marker="o", label="South")
plt.plot(months, west, marker="o", label="West")

# Title and labels
plt.title("Monthly Electricity Consumption by Neighborhood")
plt.xlabel("Month")
plt.ylabel("Electricity Consumption (kWh)")

# Grid and legend
plt.grid(True)
plt.legend()

# Display graph
plt.tight_layout()
```

plt.show()

OUTPUT:



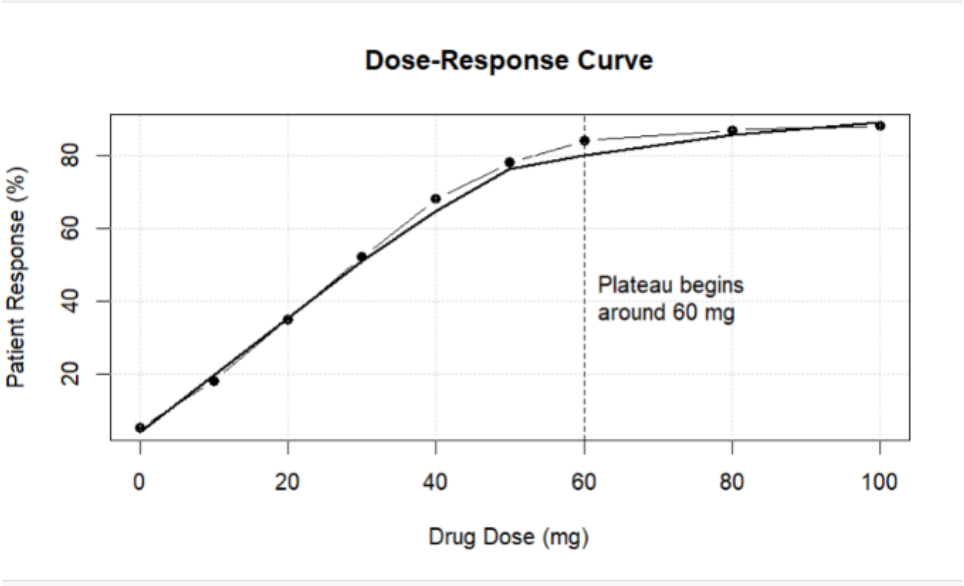
5. A pharmaceutical company is testing a new drug and needs to determine the minimum effective dose before advancing to clinical trials. Using **R**, plot a dose–response curve from the trial data (dose vs. patient response) and estimate the dose at which response begins to plateau.

CODE:

```
# Create dose-response data

dose <- c(0, 10, 20, 30, 40, 50, 60, 80, 100)

response <- c(5, 18, 35, 52, 68, 78, 84, 87, 88)


# Plot dose-response curve

plot(

  dose,

  response,
```

```
  type = "b",
  pch = 19,
  main = "Dose-Response Curve",
  xlab = "Drug Dose (mg)",
  ylab = "Patient Response (%)"
)

# Add grid
grid()

# Add smooth curve
lines(
  lowess(dose, response),
  lwd = 2
)

# Mark approximate plateau region
abline(
  v = 60,
  lty = 2
)

text(
  60,
  40,
  "Plateau begins\naround 60 mg",
  pos = 4
)
```

OUTPUT:



6. An agricultural research lab is studying how increasing fertilizer concentration affects crop yield, aiming to avoid wasteful over-application. Using **Python**, fit and plot a dose–response curve of fertilizer amount vs. yield, and advise the optimal dosage range.

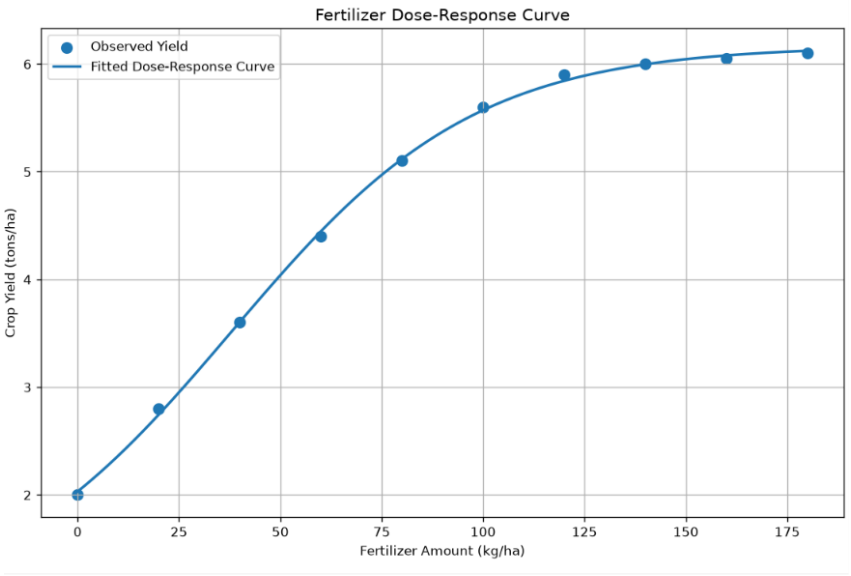
CODE:

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.optimize import curve_fit


# Fertilizer amount (kg/ha)

fertilizer = np.array([0, 20, 40, 60, 80, 100, 120, 140, 160, 180])


# Crop yield (tons/ha)

yield_data = np.array([2.0, 2.8, 3.6, 4.4, 5.1, 5.6, 5.9, 6.0, 6.05, 6.1])


# Logistic dose-response model

def dose_response(x, bottom, top, midpoint, slope):

    return bottom + (top - bottom) / (

        1 + np.exp(-(x - midpoint) / slope)
```

```
    )

# Fit the model
params, covariance = curve_fit(
    dose_response,
    fertilizer,
    yield_data,
    p0=[2, 6.2, 60, 20],
    maxfev=10000
)

# Generate smooth fitted curve
x_fit = np.linspace(0, 180, 500)
y_fit = dose_response(x_fit, *params)

# Plot original data
plt.figure(figsize=(9, 6))

plt.scatter(
    fertilizer,
    yield_data,
    s=60,
    label="Observed Yield"
)

# Plot fitted curve
plt.plot(
    x_fit,
```

```
    y_fit,
    linewidth=2,
    label="Fitted Dose-Response Curve"
)

# Labels and title
plt.xlabel("Fertilizer Amount (kg/ha)")
plt.ylabel("Crop Yield (tons/ha)")
plt.title("Fertilizer Dose-Response Curve")

plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

# Print fitted parameters
print("Estimated parameters:")
print("Bottom yield =", round(params[0], 2))
print("Maximum yield =", round(params[1], 2))
print("Midpoint dose =", round(params[2], 2), "kg/ha")
```

OUTPUT:



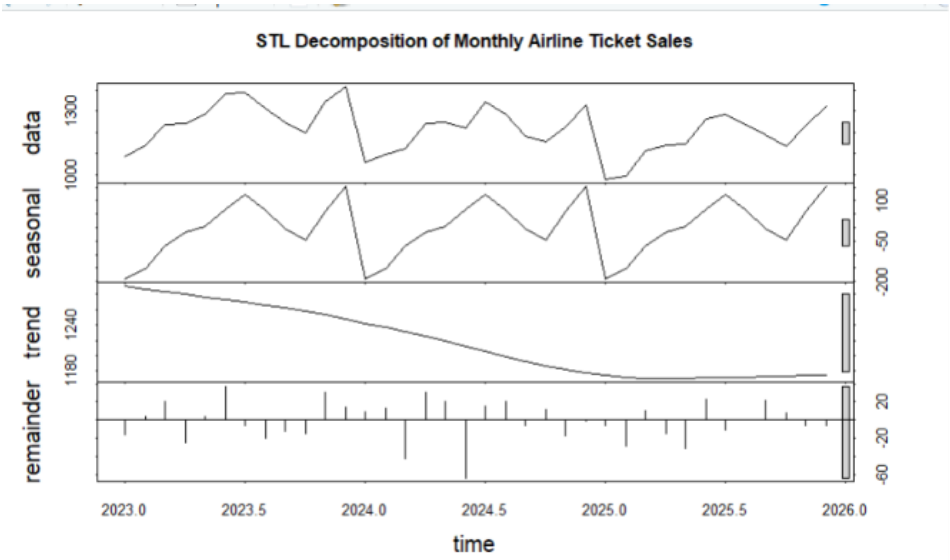
7. An airline's revenue team suspects that ticket sales follow a strong seasonal pattern tied to holidays, but a slow underlying decline is being masked by this seasonality. Using **R**, apply STL decomposition to monthly ticket sales and separate the trend, seasonal, and residual components.

CODE:

```
# Create 36 months of monthly ticket sales data

set.seed(123)


months <- seq(

  as.Date("2023-01-01"),

  as.Date("2025-12-01"),

  by = "month"

)


# Slow downward trend

trend <- seq(1200, 1050, length.out = 36)


# Seasonal pattern

seasonal_pattern <- rep(

  c(-100, -50, 0, 50, 100, 150, 200, 180, 100, 50, 150, 250),
```

```
  3
)

# Random variation
random_error <- rnorm(36, mean = 0, sd = 30)

# Monthly ticket sales
sales <- trend + seasonal_pattern + random_error

# Convert to time-series object
sales_ts <- ts(
  sales,
  start = c(2023, 1),
  frequency = 12
)

# STL decomposition
stl_result <- stl(
  sales_ts,
  s.window = "periodic"
)

# Display decomposition
plot(
  stl_result,
  main = "STL Decomposition of Monthly Airline Ticket Sales"
)
```

OUTPUT:



8. A streaming platform wants to know whether recent dips in weekly active users reflect a real decline or just normal seasonal viewing habits. Using **Python**, perform STL decomposition on the weekly user-activity data and interpret the trend component to advise the product team.

CODE:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from statsmodels.tsa.seasonal import STL


# ------------------------------------------------

# 1. Create 2 years of weekly dates

# ------------------------------------------------


dates = pd.date_range(

   start="2024-01-07",

   periods=104,

   freq="W"

)
```

```
# ------------------------------------------------
# 2. Generate weekly active users
# ------------------------------------------------

np.random.seed(42)

# Slow underlying decline
trend = np.linspace(100000, 92000, 104)

# Weekly/annual seasonal pattern
seasonal = 5000 * np.sin(
    2 * np.pi * np.arange(104) / 52
)

# Random variation
noise = np.random.normal(0, 1500, 104)

# Final user activity
users = trend + seasonal + noise

# Create DataFrame
df = pd.DataFrame({
    "Date": dates,
    "Weekly_Active_Users": users
})

# ------------------------------------------------
# 3. Convert to time series
```

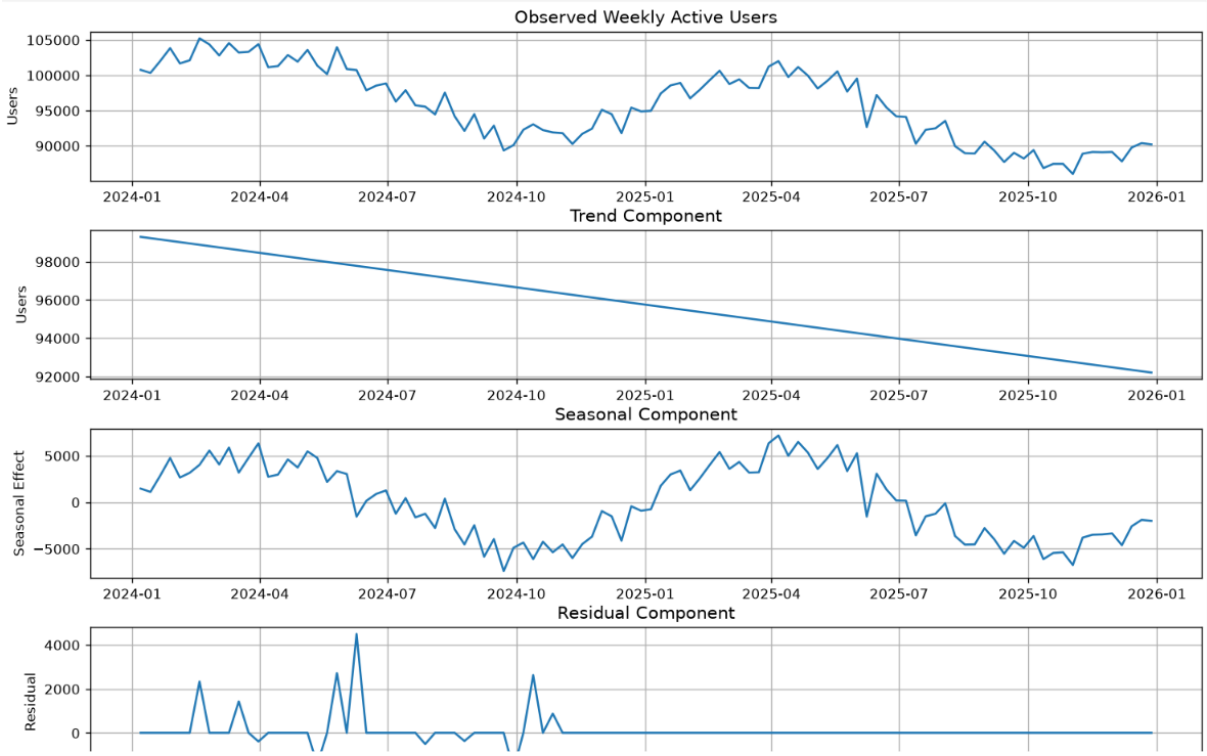
```
# ------------------------------------------------

user_ts = pd.Series(
    df["Weekly_Active_Users"].values,
    index=df["Date"]
)

# ------------------------------------------------
# 4. Apply STL decomposition
# ------------------------------------------------

stl = STL(
    user_ts,
    period=52,
    robust=True
)

result = stl.fit()

# ------------------------------------------------
# 5. Plot STL components
# ------------------------------------------------

fig, axes = plt.subplots(4, 1, figsize=(12, 10))

# Observed
axes[0].plot(user_ts)
axes[0].set_title("Observed Weekly Active Users")
```

```
axes[0].set_ylabel("Users")
axes[0].grid(True)

# Trend
axes[1].plot(result.trend)
axes[1].set_title("Trend Component")
axes[1].set_ylabel("Users")
axes[1].grid(True)

# Seasonal
axes[2].plot(result.seasonal)
axes[2].set_title("Seasonal Component")
axes[2].set_ylabel("Seasonal Effect")
axes[2].grid(True)

# Residual
axes[3].plot(result.resid)
axes[3].set_title("Residual Component")
axes[3].set_ylabel("Residual")
axes[3].set_xlabel("Date")
axes[3].grid(True)

plt.tight_layout()
plt.show()

# ------------------------------------------------
# 6. Display recent trend values
# ------------------------------------------------
```

print("Recent trend values:")

print(result.trend.tail(10))

OUTPUT:



9. A stock trading firm wants to reduce short-term noise in a volatile stock's price data to spot the underlying momentum before making buy/sell calls. Using **R**, compute and plot a 7-day and 30-day moving average over the raw price series, and explain what the crossover between them suggests.

CODE:

Load package

install.packages("zoo")

library(zoo)

Create 100 days of sample stock prices

set.seed(123)

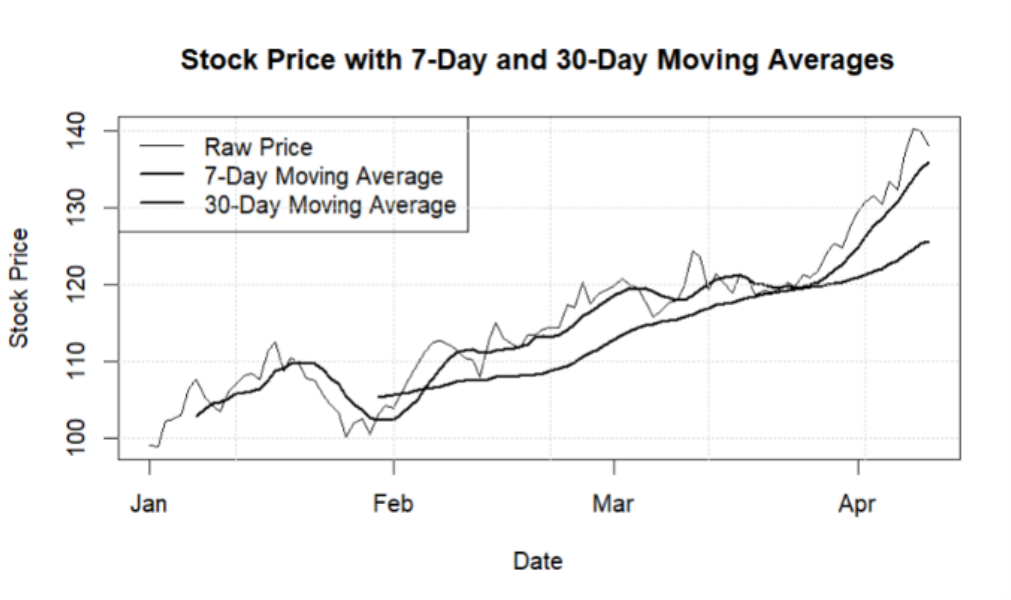
dates <- seq(

```
  as.Date("2025-01-01"),
  by = "day",
  length.out = 100
)

# Generate volatile stock prices
price <- 100 + cumsum(rnorm(100, mean = 0.2, sd = 2))

# Calculate 7-day moving average
MA7 <- rollmean(
  price,
  k = 7,
  fill = NA,
  align = "right"
)

# Calculate 30-day moving average
MA30 <- rollmean(
  price,
  k = 30,
  fill = NA,
  align = "right"
)

# Plot raw stock price
plot(
  dates,
  price,
```

```
  type = "l",
  lwd = 1,
  main = "Stock Price with 7-Day and 30-Day Moving Averages",
  xlab = "Date",
  ylab = "Stock Price"
)

# Add 7-day moving average
lines(
  dates,
  MA7,
  lwd = 2
)

# Add 30-day moving average
lines(
  dates,
  MA30,
  lwd = 2
)

# Add legend
legend(
  "topleft",
  legend = c(
    "Raw Price",
    "7-Day Moving Average",
    "30-Day Moving Average"
```

```
  ),
  lty = 1,
  lwd = c(1, 2, 2)
)


# Add grid
grid()
```

OUTPUT:



10. An economist is analyzing a country's quarterly GDP data but needs to remove the long-term growth trend to study business-cycle fluctuations in isolation. Using **Python**, detrend the GDP series (e.g., by differencing or regression-based detrending), plot the detrended series, and interpret the remaining fluctuations.

CODE:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import signal


# Create quarterly dates
```

```
dates = pd.date_range(
    start="2018-01-01",
    periods=32,
    freq="QS"
)

# Create simulated GDP data
np.random.seed(42)

# Long-term growth trend
trend = np.linspace(100, 140, 32)

# Business-cycle fluctuations
cycle = 5 * np.sin(
    2 * np.pi * np.arange(32) / 8
)

# Random noise
noise = np.random.normal(0, 1.5, 32)

# GDP = trend + cycle + noise
gdp = trend + cycle + noise

# Create DataFrame
df = pd.DataFrame({
    "Date": dates,
    "GDP": gdp
})
```

```
# ---------------------------------------
# Regression-based detrending
# ---------------------------------------

time = np.arange(len(df))

# Fit linear trend
coefficients = np.polyfit(time, df["GDP"], 1)

# Calculate fitted trend
trend_line = np.polyval(coefficients, time)

# Calculate detrended GDP
df["Detrended_GDP"] = df["GDP"] - trend_line

# ---------------------------------------
# Plot original GDP and trend
# ---------------------------------------

plt.figure(figsize=(10, 5))

plt.plot(
    df["Date"],
    df["GDP"],
    label="Original GDP",
    marker="o"
)
```

```
plt.plot(
    df["Date"],
    trend_line,
    label="Long-term Trend",
    linestyle="--"
)

plt.title("Quarterly GDP and Long-Term Trend")
plt.xlabel("Quarter")
plt.ylabel("GDP")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# ---------------------------------------
# Plot detrended GDP
# ---------------------------------------

plt.figure(figsize=(10, 5))

plt.plot(
    df["Date"],
    df["Detrended_GDP"],
    marker="o"
)
```

```
# Zero reference line
plt.axhline(
    y=0,
    linestyle="--"
)

plt.title("Detrended Quarterly GDP")
plt.xlabel("Quarter")
plt.ylabel("Deviation from Trend")
plt.grid(True)
plt.tight_layout()
plt.show()

# Display data
print(df)
```

OUTPUT: